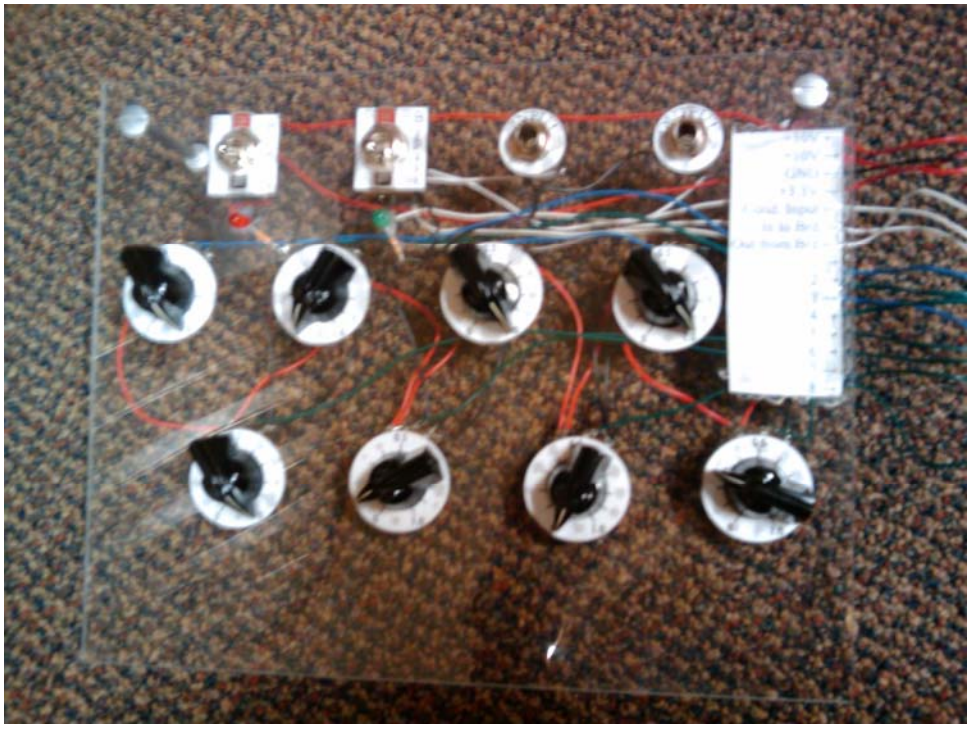
The Infinite Impulse Response Filter



by

Michael Van Antwerp

and

Nathan Jones

Table of Contents

1. Introduction 1

2. Theory 2

    2.1. Infinite Impulse Response (IIR) Filter 2

    2.2. Multiplier Accumulator (MAC) 3

3. Implementation 3

    3.1. Input/Output Conditioning 3

    3.2. Analog-to-Digital and Digital-to-Analog Converting 5

    3.3. The PIC 18F452 Microcontroller 6

    3.4. The Xilinx Spartan XC3S400 FPGA 8

4. Performance 9

    4.1. Comparison to Device Specifications 9

    4.2. Comparison to Industry 10

5. Approximations and Considerations 10

6. Conclusions 12

7. Appendices 13

    7.1. Appendix A – Full Schematic 13

    7.2. Appendix B – C Code for the PIC 14

    7.3. Appendix C – Verilog Code for the FPGA 18

Introduction

The goal of this project was to design an audio filter for use with musical instruments through a ¼" jack. More specifically, our team chose to design a filter to amplify or attenuate specific audio frequency ranges that are chosen by the user. The most efficient method of implementation was determined to be a Finite Impulse Response (FIR) or an Infinite Impulse Response (IIR) filter.

The final design presented implements an IIR filter using seven coefficients for the most recent seven input values and an additional coefficient for the last output value. These coefficients are designated through the use of eight potentiometer dials which nominally range in value from 0 to 1, although in the final implementation an effective range of 0 to 1/8 was available for each dial. These dials are sampled through the use of the PIC's onboard 10-bit analog-to-digital converter (ADC) at a rate of approximately 1 KHz for all eight dials. Additional controls available to the user are a power switch and a true bypass switch, which routes input directly to the output without any analog or digital alteration.

IIR convolution is performed after inputs are sampled with an 8-bit A/D converter. Input sampling and convolution is performed at approximately 45 KHz, the sampling rate used for CD quality audio. Outputs are synthesized with an 8-bit digital-to-analog converter (DAC). Figure 1 graphically represents the implementation of our filter, beginning with the input conditioning circuitry, proceeding to the ADC, combining with the coefficients (determined on the PIC) in the FPGA to apply the IIR, passing through a DAC, and finally going through the output conditioning circuitry.

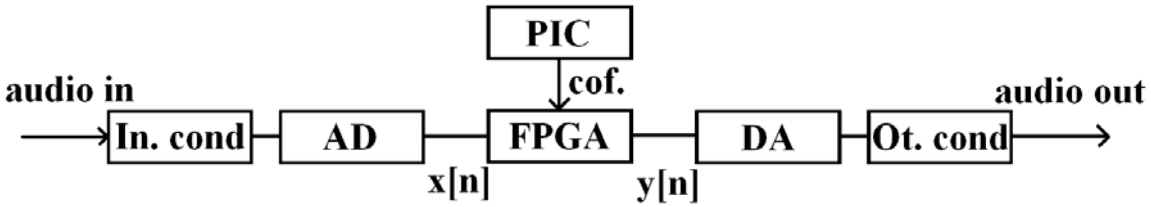


Figure 1: A graphical representation of the IIR filter, including such major components as the input/output conditioning circuitry, the ADC/DAC, and the IIR filter on the PIC/FPGA.

Coefficient sampling and data convolution is performed on the HarrisBoard 2.0 through the use of a Xilinx Spartan 3 XC3S400 FPGA and a PIC 18F452 microprocessor. The Spartan 3 contains 8,064 Logic gates for use with 116 I/O pairs and the PIC processor performs 8-bit instructions at a rate of 5 MHz with a clock speed of 20 MHz.

Theory - *Infinite Impulse Response (IIR) Filter*

An infinite impulse response filter is a system whose output is the difference equation comprised of a weighted sum of specified input values and specified output values, with the general expression as in equation 1.

$$y[n] = \sum_{q=0}^{M} x[n-q]\beta_q + \sum_{p=1}^{N} y[n-p]\alpha_p \qquad (1)$$

Here the function $y[n]$ represents the output and the function $x[n]$ represents the input. It can be shown that the transfer function for equation 1 is as in equation 2, where z is the discrete time complex frequency variable.

$$H(z) = \frac{\sum_{q=0}^{M} z^{-q}\beta_q}{\sum_{p=0}^{N} z^{-p}\alpha_p} \qquad (2)$$

From equation 2 it is obvious that the zeros of the transfer function, the frequencies where $H(z) = 0$, are the roots of the numerator and the poles of the transfer function, the

frequencies where $1/H(z) = 0$, are the roots of the denominator. Thus any transfer function can be specified by its difference equation and every difference equation has a specific transfer function. Adjusting the number and the value of the coefficients α_p and β_q will result in different transfer functions.

Theory - *Multiplier Accumulator (MAC)*

Applying the difference equation is another way of applying a multiplier accumulator (MAC). At every time step the elements in a vector of coefficients (α_p and β_q) are multiplied by the elements in a vector of data values ($y[n-1]$ to $y[n-p]$ and $x[n]$ to $x[n-q]$ respectively) and the results are added together to yield the output. On the next time step the vectors of data values are shifted one step down the proverbial assembly line and two new values, $x[n]$ and $y[n-1]$, are shifted in.

Implementation - *Input/Output Conditioning*

As important as the digital implementation was, it would do nothing without the appropriate analog interface with the instrument and amplifier. If voltage levels are not calculated correctly improper input conditioning could even destroy the components on the HarrisBoard 2.0. To properly condition the input signal the signal was first passed through a voltage follower, a simple buffer to isolate the impedances of the instrument from the impedances of the digital circuitry and prevent any loading effects. An OP07 op-amp was used to do this; a schematic is shown below in Figure A. Immediately following this stage the signal was amplified from the range -1 to 1V centered around 0V

to the range 0 – 3.3V centered around 1.65V so as to accommodate the 3.3V maximum voltage levels of LVCMOS logic. It was assumed that the incoming instrument signal would be in the range -1 to 1V based on work done one year previous by another Microprocessors team on a similar project. This conditioning was accomplished by an op-amp configured to amplify an input by 3.3/2 and add a 1.65V offset; the schematic is shown in Figure 2. Using equation 3, the general equation for the output voltage of the op-amp circuit used, it was determined what the proper resistor values would be to accomplish the task. The +3.3V regulated supply from the HarrisBoard 2.0 was used for V_{offset} because it was so strictly regulated.

$$V_{out} = V_{in}\left(-\frac{R_f}{R_1}\right) + V_{offset}\left(\frac{R_3}{R_2 + R_3}\right)\left(1 + \frac{R_f}{R_1}\right) \qquad (3)$$

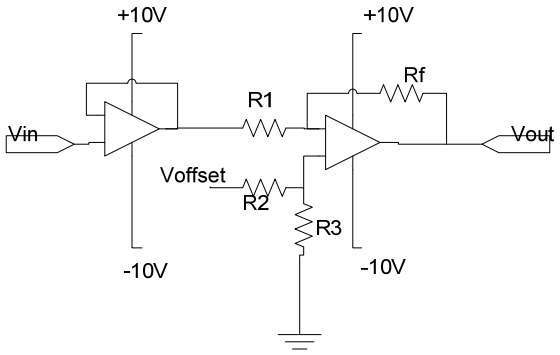


Figure 2: A general schematic showing the input conditioning circuitry used to both eliminate loading and bring the instrument voltage levels to CMOS logic levels. The upper terminal of the op-amps is the inverting terminal. To create the gain and offset levels we desired the following values were used: Voffset = +3.3V, R1 = 200 Ω, R2 = 620 Ω, R3 = 150 Ω, Rf = 330 Ω.

The output was first sent to a voltage follower in the same setup as in Figure 2, again to isolate the device from any other loads (such as guitar amplifiers). After this stage the output signal was passed to a 2nd-order Butterworth low-pass filter using a Sallen-Key. Values for the components were chosen such that the cutoff or half-power

point was at approximately 21 kHz; this was to filter out any high frequency noises in our output such as power noise or thermal noise and only keep the frequencies within the audio range. The schematic for the output conditioning circuitry is shown in Figure 3.

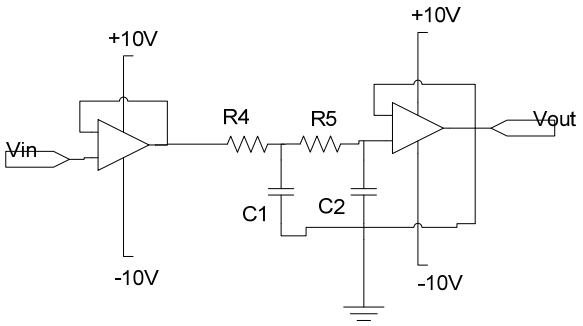


Figure 3: A schematic showing the output conditioning circuitry: a buffer and a 2nd-order low-pass Butterworth filter. To create a cutoff frequency at 21 kHz the following values were used: C1 = 0.0068 μF, C2 = 0.015 μF, R4 = R5 = 750 Ω.

Experiments proved that the output signal was highly degraded by noise; such noises made the actual signal barely intelligible. The addition of the low-pass filter made the output, while still very noisy, an obvious attempt to filter and process an instrument signal.

Implementation - *Analog-to-Digital and Digital-to-Analog Converting*

Before the conditioned signal could be processed it would need to be digitized. An ADC made by Texas Instruments (manufacturers part no. TLV0831) was used to accomplish this task. The TLV0831 is a serial ADC that, after receiving two clock cycles, first outputs the most significant bit (MSB) of an 8-bit digital approximation followed by the rest of the number. It requires a total of 10 clock cycles to output one complete 8-bit number from the TLV0831. Control programming on the PIC provided the clock and control signals such that the TLV0831 output an 8-bit data value at a frequency of about

45 kHz. Each result was shifted serially into the FPGA which used eight parallel registers to go from the serial ADC data line to the parallel FPGA MAC. The TLV0831 recognized standard LVCMOS logic levels and was designed to be powered from a +2.7-3.6V supply. Care was taken to make sure operation was within the recommended operating conditions such as ensuring the provided clock was within the range 20-600 kHz, an operating restriction on the TLV0831. A schematic showing the TLV0831 as it was wired in the final circuit is shown in Appendix A.

After the analog signal was digitized and processed by the FPGA it was necessary to convert it back to an analog signal suitable for use in other standard audio applications. A DAC made by Analog Devices (manufacturers part no. AD558) was used to accomplish this task. Since the output data was being buffered on the FPGA by an output register the AD558 was wired as transparent, receiving no control commands and converting to an analog voltage any digital signal at its 8-bit wide parallel input. The AD558 recognized standard LVCMOS voltage levels and was designed to be powered from a +5-15V power supply. A schematic showing the AD558 as it was wired in the final circuit is shown in Appendix A.

Implementation - *The PIC 18F452 Microcontroller*

As it was used the PIC served two basic functions: to operate the control signals that would facilitate the transfer of information to, through, and out of the FPGA and to digitize and update the difference equation coefficients and send them to the FPGA. The heart of the PIC control code was the PWM module on the PIC which used TIMER2 and a register called PR2 to create a configurable pulse train. In wanting to get CD quality

sampling or greater (44.1 kHz) from our filter the PWM was configured to create a pulse train at 450 kHz with a duty cycle of 45%. Since it took 10 clock cycles to produce one 8-bit output value from the TLV0831 a clock at 450 kHz produced an output value at a rate of 45 kHz. The duty cycle on the pulse train was designed to stay within the operation restriction on the TLV0831 that the duty cycle of any clock signal must be in the range 40-60%. The pulse train was also connected to T0CKI, the input pin that TIMER0 used as a clock. TIMER0 counted the pulses of the pulse train and was configured such that every ten pulses the serial-to-parallel register on the FPGA shifted its data (the full 8-bit result of an AD conversion from the TLV0831) into an 8-bit parallel register as part of the data chain. This signal also triggered the rest of the 8-bit parallel registers in the FPGA to shift their data values down one register thus implementing the MAC discussed in the Theory section.

The coefficients of the difference equation were set via an array of eight potentiometers connected as voltage dividers such that their each of their outputs swung from V_{dd} to ground. Each voltage was connected to one of the eight analog inputs used by the PIC's onboard ADC. In the code the PIC periodically iterates through each of the eight channels, acquiring a 10-bit digital value representing the position of each potentiometer. Should the new set of coefficient values be different than the ones that were in memory at the time, the PIC updated its coefficient array with the new values and sent the new values to the FPGA. The PIC did this by repeatedly placing the coefficients in parallel across ten output pins and then managing a control signal that told the FPGA when to shift in new values to its 10-bit parallel coefficient registers.

Implementation - *The Xilinx Spartan XC3S400 FPGA*

Data convolution is performed in parallel on the Spartan 3 to increase processing speed and free the PIC to process coefficients. Input data bits are loaded from the A/D converter into serial-to-parallel registers, and then used as 8-bit parallel values and loaded into registers. The entire sampling process is performed with an approximately 45 KHz frequency. The most recent output value is also stored in an 8-bit register for use according to the IIR convolution algorithm. Figure 4, below, is a graphical representation of the MAC implemented on the FPGA. The top row of registers, multipliers, and adders represents the MAC operation on the input data. The registers hold the 8-bit data values and represent the vector $x[n]$ to $x[n-q]$, with each value being multiplied by the vector of coefficients b0 to bq (b3 in the figure) and added together. The bottom row represents the MAC operation on the output data. The registers hold the 8-bit data values and represent the vector $y[n-1]$ to $y[n-p]$, with each value being multiplied by the vector of coefficients a0 to ap (a3 in the figure) and added together. The final result from both MACs is added together and sent to the output.

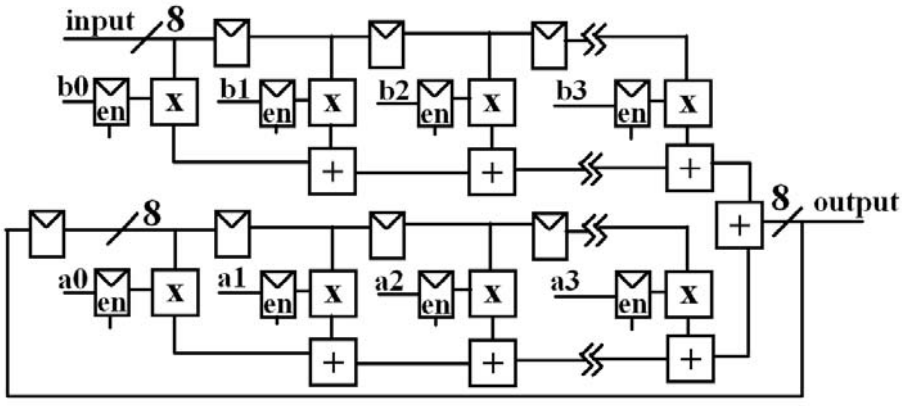


Figure 4: A representation of the MAC architecture implemented on the FPGA; the upper row represents the MAC of the input signal, x, with the x coefficients (β_0 to β_q) and the bottom row represents the MAC of the output signal, y, with the y coefficients (α_1 to α_p).

Convolution coefficients are received from the PIC along with a correlated clock and reset signal. Upon reset, convolution coefficients received are loaded into 10-bit registers individually in order on each clock rising edge. Coefficients are loaded in this manner instead of implementing a simple serial register load in order to minimize glitch time during coefficient loads.

Loaded coefficient and input/output values are multiplied together using a shift adder algorithm directly implemented in hardware. The eight multiplication results are then added together by using three rows of adders implemented in parallel. During this process, the bit-width of the signal grows in size due to the fact that the result of a multiplication or addition operation has a larger range than the individual inputs. After completion of the IIR calculations, the most significant eight bits of the result are sent to the DAC and the less significant bits are discarded. As a consequence of this process and the fact that the output range is equal to the input range, the effective maximum coefficient 10 bit value of 0x3FF results in an output of 1/8 of the input, assuming none of the most significant bits are discarded.

Performance - *Comparison to Device Specifications*

The first and most basic goal, to build a device that applies an IIR filter to an input signal, was successfully met. The proof of this fact was established by applying a 22.5 kHz sine wave to the input of our filter. Because the device was sampling the input at 45 kHz, each coefficient of the difference equation was separated in the time domain by $1/45kHz = 22.2\mu s$ which is exactly one-half of a cycle of a 22.5 kHz sine wave. Thus one would predict that should this sine wave be the input to the filter, increasing every other

coefficient would be the same thing as adding the wave shifted by a full cycle which creates constructive interference. Similarly increasing two adjacent coefficients would be the same thing as adding the wave shifted by one-half of a cycle which creates destructive interference. Both of these phenomena were observed during experimentation of the device.

Performance – *Comparison to Industry*

Unfortunately the device was plagued with an exorbitant amount of noise, so much so that it was only after the application of the low-pass filter that the output signal was even heard above the noise floor. While there are many possible reasons for the introduction of this noise into our system (power noise and thermal noise for example) the quick conclusion is that such a device would not last a day in industry. It degrades the very quality that it is supposed to preserve and enhance! The IIR filter, while a nice proof of concept that applicable, powerful digital signal processing is available to small-scale embedded system developers, is nonetheless a failed attempt to develop a market competitive device.

Approximations and Considerations

Before the report is concluded it would be pertinent to present a brief discussion of the approximations that were made during development in addition to any recommendations for future implementations.

One of the first things the team would have like to have seen implemented was a better power switch. As it was wired the current switch only broke the connection to the +10V

power supply and not the -10V power supply. While this did effectively turn off the HarrisBoard 2.0 and thus the filter, it meant that the op-amps could still draw power when the device was "off." Such a consequence of the design could mean unnecessary power leakage when the device is not in operation.

Second it was noticed that the input conditioning circuitry designed earlier did not produce an output in the range 0-3.3V as was expected. The actual range was roughly 0-2.7V and constituted a degradation in the resolution of the ADC since "full-scale" became the 8-bit value for 2.7/3.3 and not the value 255 as was expected. Better care should be taken for the input conditioning circuitry to ensure as little information about the signal is lost.

Third, as has been mentioned several times, the device was troubled with much noise. While the causes of this noise are not entirely known, a general recommendation for future implementations would be to spend a good amount of time designing the analog circuitry to eliminate as much noise as is feasibly possible.

In addition to this future implementations should consider the significance of the bits in the output of the FPGA. While eliminating bits removes information about the output signal many of the output bits provide unnecessary resolution or unnecessary maximum values. The 8-bits that seemed most likely to contain the majority of the information about the output were chosen by quick experimentation. Future developers should consider a more rigorous method of determining which bits to keep.

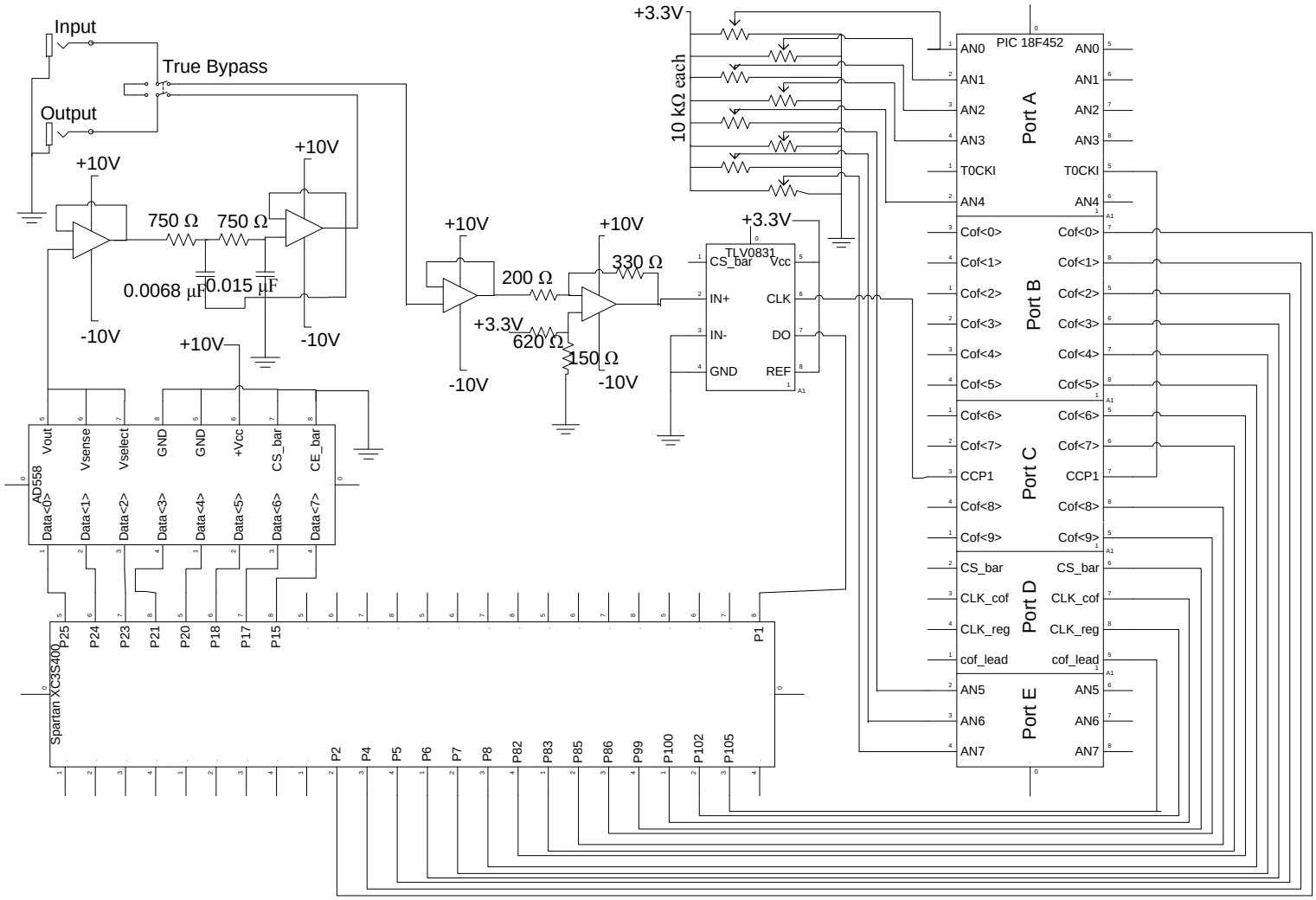
Lastly future implementations should consider the design of the filter as a whole and consider whether building an IIR filter with eight coefficients has much of an effect on an audio signal sampled at 45 kHz. All of the effects of the current filter were negated by the

noise in the circuit; a better overall design might choose an implementation that has a greater significance on the range of frequently used audio frequencies.

Conclusions

Overall the team is very satisfied with the results of the project and proud of the fact that, at least to a small extent, it represents an industry-standard audio filtering device. FPGAs in general can be very well suited for high-speed, computationally intensive applications and this was taken advantage of by programming it to be the MAC. The PIC's onboard modules gave the device the versatility and control signals it needed to function properly. The analog circuitry, while in need of improvement, gave the device just enough analog processing power to produce an audible output. On several levels, however, the project was a learning experience by making mistakes, not the least of which was to try to create a project beyond the scope of the team's capabilities. To a nearly similar extent on each of the major components (the PIC, the FPGA, and the analog circuitry) not enough time was spent maximizing each component's specifications. Future projects will consider the complexity of each of a device's components and focus on producing a few quality components rather than many mediocre components.

Appendix A - *Full Schematic*



Appendix B – *C Code for the PIC*

```
#include <p18f452.h>
#include <timers.h>

/*Ports
A0: AN0
A1: AN1
A2: AN2
A3: AN3
A4: T0CKI
A5: AN4
A6: NA
A7: NA

B: Cofi[0:5]

C0: Cofi[6]
C1: Cofi[7]
C2: CCP1
C3: Cofi[8]
C4: Cofi[9]

D0: NA
D1: NA
D2: CS_bar
D3: CLK_cof
D4: CLK_reg
D5: NA
D6: NA
D7: cof_load

E0: AN5
E1: AN6
E2: AN7
*/

// Function Prototypes
void high_interrupt(void);
void main(void);
void isr(void);
void initialize(void);
void poll_sliders(void);
int equal_cof(void);
void update_cof(void);
void send_cof(void);

// Establish high priority interrupts
#pragma code high_vector = 0x08
void high_interrupt(void){
   _asm
      GOTO isr
   _endasm
}

// Begin main code
#pragma code

// Global Variables
int Cof[8] = {0x3FF,0,0,0,0,0,0,0};
int Cof_temp[8];
```

```
void main(void){
   /*The main program begins by initializing all pertinent registers
   and ports to set up Timer 0, the PWM module and the ADC module.
   It then enters an infinite loop that first polls the potentiometers
   for the coefficient values. If the values are the same as the
   previous values nothing happens. If they are different then the
   array storing the current values is updated and sent to the FPGA.*/
   initialize();
   while(1){
      poll_sliders();
      if (!equal_cof()) {
         update_cof();
         send_cof();
      }
   }
}

void initialize(void){
   /*Initialize Timer 0, the counter that controls when the data
   registers in the FPGA update, which is set to trigger a pulse
   every ten pulses of the PWM, corresponding to the ADC outputting
   a full 8-bit value. Set TMR0L to be F7h so that it interrupts
   after ten pulses of the PWM. Sets INTCON to enable global
   interrupts.*/
   T0CON = 0b11101000;
   TMR0L = 0xF7;
   INTCON = 0b10100000;

   /*Initialize the PWM. PR2 = 10 means the output will have
   a period of 2.2 us. CCPR1L = 5 means the output
   will have a duty cycle of 45%.*/
   PR2 = 0x0A;
   CCPR1L = 0b00000101;
   T2CON = 0b00000100;
   CCP1CON = 0b00001100;

   /*Initialize functions for the port pins being used by
   the ADC.*/
   ADCON1 = 0b10000000;

   /*Sets port states. Port A and port E are being
   used as inputs for the ADC; the fourth port of A (RA4)
   is being used as the clock input for Timer 0. Port D
   is being used to output the two pseudo-clock signals
   and pertinent data signals to the ADC and the FPGA.
   Port B and part of Port C are being used to transfer
   the coefficients to the FPGA in parallel. One port on
   Port C (RC2) is used the output of the PWM and used as
   the CLK for the ADC.*/
   TRISA = 0b11111100;
   TRISB = 0b00000000;
   TRISC = 0b00000000;
   TRISD = 0b00000000;
   TRISE = 0b00000111;

   /*The third pin of Port D is pulled high then low to
   initiate the sampling of the input signal.*/
   PORTD = 0b01000100;
   PORTD = 0b01000000;

   send_cof();
}
```

```
void poll_sliders(void){
   /*Repeatedly updates the ADCON0 register to read from each of the
   8 channels, waits for the conversion to complete, then puts the
   result into an array holding the temporary coefficients (Cof_temp).*/
   int i = 0;
   int j = 0;

   //Channel 0
   ADCON0 = 0b10000001;
   for( j = 0; j<20; j++);
   ADCON0 = 0b10000101;
   while(0b00000100 & ADCON0);
   Cof_temp[i] = ADRESH;
   Cof_temp[i] = Cof_temp[i]<<8;
   Cof_temp[i++] = Cof_temp[i] | ADRESL;

   //Channel 1
   ADCON0 = 0b10001001;
   for( j = 0; j<20; j++);
   ADCON0 = 0b10001101;
   while(0b00000100 & ADCON0);
   Cof_temp[i] = ADRESH;
   Cof_temp[i] = Cof_temp[i]<<8;
   Cof_temp[i++] = Cof_temp[i] | ADRESL;

   //Channel 2
   ADCON0 = 0b10010001;
   for( j = 0; j<20; j++);
   ADCON0 = 0b10010101;
   while(0b00000100 & ADCON0);
   Cof_temp[i] = ADRESH;
   Cof_temp[i] = Cof_temp[i]<<8;
   Cof_temp[i++] = Cof_temp[i] | ADRESL;

   //Channel 3
   ADCON0 = 0b10011001;
   for( j = 0; j<20; j++);
   ADCON0 = 0b10011101;
   while(0b00000100 & ADCON0);
   Cof_temp[i] = ADRESH;
   Cof_temp[i] = Cof_temp[i]<<8;
   Cof_temp[i++] = Cof_temp[i] | ADRESL;

   //Channel 4
   ADCON0 = 0b10100001;
   for( j = 0; j<20; j++);
   ADCON0 = 0b10100101;
   while(0b00000100 & ADCON0);
   Cof_temp[i] = ADRESH;
   Cof_temp[i] = Cof_temp[i]<<8;
   Cof_temp[i++] = Cof_temp[i] | ADRESL;

   //Channel 5
   ADCON0 = 0b10101001;
   for( j = 0; j<20; j++);
   ADCON0 = 0b10101101;
   while(0b00000100 & ADCON0);
   Cof_temp[i] = ADRESH;
   Cof_temp[i] = Cof_temp[i]<<8;
   Cof_temp[i++] = Cof_temp[i] | ADRESL;

   //Channel 6
   ADCON0 = 0b10110001;
```

```
   for( j = 0; j<20; j++);
   ADCON0 = 0b10110101;
   while(0b00000100 & ADCON0);
   Cof_temp[i] = ADRESH;
   Cof_temp[i] = Cof_temp[i]<<8;
   Cof_temp[i++] = Cof_temp[i] | ADRESL;

   //Channel 7
   ADCON0 = 0b10111001;
   for( j = 0; j<20; j++);
   ADCON0 = 0b10111101;
   while(0b00000100 & ADCON0);
   Cof_temp[i] = ADRESH;
   Cof_temp[i] = Cof_temp[i]<<8;
   Cof_temp[i++] = Cof_temp[i] | ADRESL;
}

int equal_cof(void){
   /*Checks to see if the coefficients are equal to each other
   by iterating through a for loop.*/
   int new_cof = 1;
   int i;
   for ( i=0; i < 8; i++){
      if (Cof[i] != Cof_temp[i]){
         new_cof = 0;
         break;
      }
   }
}

void update_cof(void){
   /*Sets the array holding the current values for the coefficients
   (Cof) equal to the array holding the temporary coefficients (Cof_temp).*/
   int i;
   for ( i=0; i < 8; i++){
      Cof[i] = Cof_temp[i];
   }
}

void send_cof(void){
   /*Sends the coefficients to the FPGA by placing each one in succession
   to Port B and part of Port C in parallel and creating a psuedo-clock
   that will update the registers on the FPGA. RD7 resets an FSM on the
   FPGA to facilitate the transfer of the coefficients.*/
   int i;
   PORTDbits.RD7 = 1;
   PORTDbits.RD7 = 0;
   for ( i = 0; i < 8; i++){
      PORTB = 0b0000000000111111 & Cof[i];
      PORTCbits.RC0 = (0b0000000001000000 & Cof[i])>>6;
      PORTCbits.RC1 = (0b0000000010000000 & Cof[i])>>7;
      PORTCbits.RC3 = (0b0000000100000000 & Cof[i])>>8;
      PORTCbits.RC4 = (0b0000001000000000 & Cof[i])>>9;
      PORTDbits.RD3 = 0;
      PORTDbits.RD3 = 1;
   }
}

// Begin interrupt code
#pragma interrupt isr
void isr(void){
   /*Resets Timer 0. Finishes the off-board ADC conversion by pulling
   RD2 high then updates the registers in the FPGA holding the data
```

```
   values by creating a psuedo-clock on RD4. The next ADC conversion
   is begun by pulling RD2 low again.*/
   INTCONbits.TMR0IF = 0;
   TMR0L = 0xF7;
   PORTDbits.RD2 = 1;
   PORTDbits.RD4 = 1;
   PORTDbits.RD2 = 0;
   PORTDbits.RD4 = 0;
}
```

Appendix C – *Verilog Code for the FPGA*

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    22:12:18 11/28/2008
// Design Name:
// Module Name:    IIR_Filter
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: Convolves eight 8-bit signal data points from a serial A/D
//              converter with eight 12-bit coefficients from the PIC and outputs
//              8-bits selected from the result.
//
// Dependencies: mult12x8.v,Adder12b.v,Adder20b.v,Adder21b.v,
//               Adder22b.v,splitter.v,Full_Adder.v,coef_clk_fsm.v
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module IIR_Filter(
    input      [9:0] coef_in,             // Convolution coefficients from PIC
    input            Xser,                // Serial data input
    input            clk, SPclk, reset,   // Reg and ser-to-par clk and reset
    input            C_control, C_reset,  // Control bits for loading coefficients
    input      [3:0] SW,                  // Dip switchs used for debugging
    output     [7:0] Y,                   // Parallel data output
    output reg [9:0] test                 // Output bits used for debugging
    );

      wire [7:0] Xpar;                    // Parallel input
      reg [7:0] X,X1,X2,X3,X4,X5,X6,Y1;   // Signal registers
      reg [11:0] coef,coef1,coef2,coef3,  // Coefficient registers
                 coef4,coef5,coef6,coef7;
      wire [7:0] C_clocks;                // Clks for coefficient registers

      wire [19:0] MOut0,MOut1,MOut2,      // Outputs from Multiplication modules
                  MOut3,MOut4,MOut5,
                  MOut6,MOut7;
      wire [20:0] AOut01,AOut02,          // Outputs from first level adders
                  AOut03,AOut04;
      wire [21:0] AOut11,AOut12;          // Outputs from second level adders
      wire [22:0] AOut21;                 // Output from final adder module


      assign Y = Y1;                      // Assign output
      ser_to_par SP(Xser,SPclk,reset,Xpar);
      coef_clk_fsm clocker(C_control,C_reset,C_clocks);


      always@(posedge clk or posedge reset) begin
         if (reset) begin
            X <= 0;                       // Signal register reset logic
            X1 <= 0;
            X2 <= 0;
            X3 <= 0;
            X4 <= 0;
            X5 <= 0;
            X6 <= 0;
            Y1 <= 0;
         end else begin
            X <= Xpar;                    // Signal register logic
            X1 <= X;
            X2 <= X1;
            X3 <= X2;
            X4 <= X3;
```

```
      X5 <= X4;
      X6 <= X5;
      Y1  <= AOut21[22:15];         // Output bits selected from final adder
   end
end


/* Here each coefficient is given a register with an individual clock signal
   based on a clock splitting FSM in order to decrease glitch time during
   coefficient loads */
always@(posedge C_clocks[0] or posedge reset)
   if (reset)
      coef  <= 0;
   else
      coef  <= { coef_in, 2'b0 };

always@(posedge C_clocks[1] or posedge reset)
   if (reset)
      coef1  <= 0;
   else
      coef1  <= { coef_in, 2'b0 };

always@(posedge C_clocks[2] or posedge reset)
   if (reset)
      coef2  <= 0;
   else
      coef2  <= { coef_in, 2'b0 };

always@(posedge C_clocks[3] or posedge reset)
   if (reset)
      coef3  <= 0;
   else
      coef3  <= { coef_in, 2'b0 };

always@(posedge C_clocks[4] or posedge reset)
   if (reset)
      coef4  <= 0;
   else
      coef4  <= { coef_in, 2'b0 };

always@(posedge C_clocks[5] or posedge reset)
   if (reset)
      coef5  <= 0;
   else
      coef5  <= { coef_in, 2'b0 };

always@(posedge C_clocks[6] or posedge reset)
   if (reset)
      coef6  <= 0;
   else
      coef6  <= { coef_in, 2'b0 };

always@(posedge C_clocks[7] or posedge reset)
   if (reset)
      coef7  <= 0;
   else
      coef7  <= { coef_in, 2'b0 };


/* 12- by 8- bit multiplier modules are used to multiply each signal value by
   its respective coefficient */
mult12x8 mult0(coef,X,MOut0);
mult12x8 mult1(coef1,X1,MOut1);
mult12x8 mult2(coef2,X2,MOut2);
mult12x8 mult3(coef3,X3,MOut3);
mult12x8 mult4(coef4,X4,MOut4);
mult12x8 mult5(coef5,X5,MOut5);
mult12x8 mult6(coef6,X6,MOut6);
mult12x8 mult7(coef7,Y1,MOut7);
```

```
      /* Three successive levels of adders are used to combine the weighted signals */
      Adder_20b A01(MOut0,MOut1,AOut01);   // First adder level
      Adder_20b A02(MOut2,MOut3,AOut02);
      Adder_20b A03(MOut4,MOut5,AOut03);
      Adder_20b A04(MOut6,MOut7,AOut04);
      Adder_21b A11(AOut01,AOut02,AOut11); // Second adder level
      Adder_21b A12(AOut03,AOut04,AOut12);
      Adder_22b A21(AOut11,AOut12,AOut21); // Final adder level


      /* This code is used to output results based on dip switch positions in order
         to aid in debugging the code */
      always@(*)
         case (SW)
            4'b0000: test <= {coef[11:3], C_control};
            4'b0001: test <= {coef1[11:3], C_control};
            4'b0010: test <= {coef2[11:3], C_control};
            4'b0011: test <= {coef3[11:3], C_control};
            4'b0100: test <= {coef4[11:3], C_control};
            4'b0101: test <= {coef5[11:3], C_control};
            4'b0110: test <= {coef6[11:3], C_control};
            4'b0111: test <= {coef7[11:3], C_control};
            4'b1000: test <= {X,2'b0};
            4'b1001: test <= {X1,2'b0};
            4'b1010: test <= {X2,2'b0};
            4'b1011: test <= {X3,2'b0};
            4'b1100: test <= {Y1,2'b0};
            4'b1101: test <= {C_clocks,2'b0};
            default: test <= 10'b0;
         endcase

endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    23:57:19 12/01/2008
// Design Name:
// Module Name:    ser_to_par
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: turns a 1-bit serial signal into an 8-bit parallel signal
//
// Dependencies: none
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module ser_to_par(
    input ser,
    input clk,
    input reset,
    output reg [7:0] par
    );

   always@(posedge clk or posedge reset)begin
      if (reset) begin
         par[0] <= 0;      // reset logic
         par[1] <= 0;
         par[2] <= 0;
         par[3] <= 0;
         par[4] <= 0;
         par[5] <= 0;
         par[6] <= 0;
         par[7] <= 0;
      end else begin       // serial register loading for parallel output
```

```
         par[0] <= ser;
         par[1] <= par[0];
         par[2] <= par[1];
         par[3] <= par[2];
         par[4] <= par[3];
         par[5] <= par[4];
         par[6] <= par[5];
         par[7] <= par[6];
       end
   end

endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    08:56:28 12/09/2008
// Design Name:
// Module Name:    coef_clk_fsm
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: This module uses an FSM to split the first eight control signals
//              after reset into separate bits. This is used to load convolution
//              coefficients directly into registers instead of loading them
//              serially, which drastically reduces glitch time.
//
// Dependencies: none
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module coef_clk_fsm(
    input clk,
    input reset,
    output reg [7:0] clks
    );

    reg [8:0] count;                    // counting variable

    /* This counter starts at 0 and increments upon each clock signal. Upon
       overflow the counter resets to 15 and upon reset it returns to 0 */
    always @(posedge clk or posedge reset)
      if (reset)                        //reset logic
         count <= 0;
      else begin
         count <= count+1;              //counter
         if (count == 254) begin        //end of count logic
            count <= 15;
         end
      end

    /* Combinational logic determines the control clock signals and sets them
       through a register to minimize timing delays */
    always@(posedge clk)
         case(count)
            0: clks <= 8'b00000001;
            1: clks <= 8'b00000010;
            2: clks <= 8'b00000100;
            3: clks <= 8'b00001000;
            4: clks <= 8'b00010000;
            5: clks <= 8'b00100000;
            6: clks <= 8'b01000000;
            7: clks <= 8'b10000000;
            default: clks <= 0;
         endcase
```

```
endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    22:55:21 12/01/2008
// Design Name:
// Module Name:    mult12x8
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: Multiplies a 12-bit value with an 8-bit value, resulting in a
//              20-bit value. This is completed using eight rows of 13-bit adders.
//
// Dependencies: Adder_13b.v, Full_Adder.v
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module mult12x8(
    input [11:0] A,
    input [7:0] B,
    output [19:0] M
    );

    wire [11:0] B0,B1,B2,B3,B4,B5,B6,B7;
    wire [12:0] C1,C2,C3,C4,C5,C6;

    splitter S0(B[0],B0);
    splitter S1(B[1],B1);
    splitter S2(B[2],B2);
    splitter S3(B[3],B3);
    splitter S4(B[4],B4);
    splitter S5(B[5],B5);
    splitter S6(B[6],B6);
    splitter S7(B[7],B7);

    Adder_13b A1( {1'b0, B0&A}, {B1&A, 1'b0}, {C1,M[0]} );
    Adder_13b A2( C1, {B2&A, 1'b0}, {C2,M[1]} );
    Adder_13b A3( C2, {B3&A, 1'b0}, {C3,M[2]} );
    Adder_13b A4( C3, {B4&A, 1'b0}, {C4,M[3]} );
    Adder_13b A5( C4, {B5&A, 1'b0}, {C5,M[4]} );
    Adder_13b A6( C5, {B6&A, 1'b0}, {C6,M[5]} );
    Adder_13b A7( C6, {B7&A, 1'b0}, M[19:6] );

endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    20:54:04 11/28/2008
// Design Name:
// Module Name:    Adder_20b
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: Adds two 20-bit fixed-point numbers, outputting a 21-bit value.
//
// Dependencies: Full_Adder.v
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
```

```
module Adder_20b(
    input [19:0] A,
    input [19:0] B,
    output [20:0] S
    );

      wire c1, c2, c3, c4, c5, c6, c7, c8, c9, c10, c11,
           c12, c13, c14, c15, c16, c17, c18, c19;

      Full_Adder FA1(A[0], B[0], 1'b0, S[0], c1);
      Full_Adder FA2(A[1], B[1], c1, S[1], c2);
      Full_Adder FA3(A[2], B[2], c2, S[2], c3);
      Full_Adder FA4(A[3], B[3], c3, S[3], c4);
      Full_Adder FA5(A[4], B[4], c4, S[4], c5);
      Full_Adder FA6(A[5], B[5], c5, S[5], c6);
      Full_Adder FA7(A[6], B[6], c6, S[6], c7);
      Full_Adder FA8(A[7], B[7], c7, S[7], c8);
      Full_Adder FA9(A[8], B[8], c8, S[8], c9);
      Full_Adder FA10(A[9], B[9], c9, S[9], c10);
      Full_Adder FA11(A[10], B[10], c10, S[10], c11);
      Full_Adder FA12(A[11], B[11], c11, S[11], c12);
      Full_Adder FA13(A[12], B[12], c12, S[12], c13);
      Full_Adder FA14(A[13], B[13], c13, S[13], c14);
      Full_Adder FA15(A[14], B[14], c14, S[14], c15);
      Full_Adder FA16(A[15], B[15], c15, S[15], c16);
      Full_Adder FA17(A[16], B[16], c16, S[16], c17);
      Full_Adder FA18(A[17], B[17], c17, S[17], c18);
      Full_Adder FA19(A[18], B[18], c18, S[18], c19);
      Full_Adder FA20(A[19], B[19], c19, S[19], S[20]);

endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    21:20:20 11/28/2008
// Design Name:
// Module Name:    Adder_21b
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: Adds two 21-bit numbers, resulting in a 22-bit value.
//
// Dependencies: Full_Adder.v
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module Adder_21b(
    input [20:0] A,
    input [20:0] B,
    output [21:0] S
    );

      wire c1, c2, c3, c4, c5, c6, c7, c8, c9, c10, c11,
           c12, c13, c14, c15, c16, c17, c18, c19, c20;

      Full_Adder FA1(A[0], B[0], 1'b0, S[0], c1);
      Full_Adder FA2(A[1], B[1], c1, S[1], c2);
      Full_Adder FA3(A[2], B[2], c2, S[2], c3);
      Full_Adder FA4(A[3], B[3], c3, S[3], c4);
      Full_Adder FA5(A[4], B[4], c4, S[4], c5);
      Full_Adder FA6(A[5], B[5], c5, S[5], c6);
      Full_Adder FA7(A[6], B[6], c6, S[6], c7);
      Full_Adder FA8(A[7], B[7], c7, S[7], c8);
      Full_Adder FA9(A[8], B[8], c8, S[8], c9);
      Full_Adder FA10(A[9], B[9], c9, S[9], c10);
```

```
      Full_Adder FA11(A[10], B[10], c10, S[10], c11);
      Full_Adder FA12(A[11], B[11], c11, S[11], c12);
      Full_Adder FA13(A[12], B[12], c12, S[12], c13);
      Full_Adder FA14(A[13], B[13], c13, S[13], c14);
      Full_Adder FA15(A[14], B[14], c14, S[14], c15);
      Full_Adder FA16(A[15], B[15], c15, S[15], c16);
      Full_Adder FA17(A[16], B[16], c16, S[16], c17);
      Full_Adder FA18(A[17], B[17], c17, S[17], c18);
      Full_Adder FA19(A[18], B[18], c18, S[18], c19);
      Full_Adder FA20(A[19], B[19], c19, S[19], c20);
      Full_Adder FA21(A[20], B[20], c20, S[20], S[21]);

endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    23:21:49 12/01/2008
// Design Name:
// Module Name:    Adder_22b
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: Adds two 22-bit numbers, resulting in a 23-bit value.
//
// Dependencies: Full_Adder.v
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module Adder_22b(
    input [21:0] A,
    input [21:0] B,
    output [22:0] S
    );

      wire c1, c2, c3, c4, c5, c6, c7, c8, c9, c10, c11,
           c12, c13, c14, c15, c16, c17, c18, c19, c20, c21;

      Full_Adder FA1(A[0], B[0], 1'b0, S[0], c1);
      Full_Adder FA2(A[1], B[1], c1, S[1], c2);
      Full_Adder FA3(A[2], B[2], c2, S[2], c3);
      Full_Adder FA4(A[3], B[3], c3, S[3], c4);
      Full_Adder FA5(A[4], B[4], c4, S[4], c5);
      Full_Adder FA6(A[5], B[5], c5, S[5], c6);
      Full_Adder FA7(A[6], B[6], c6, S[6], c7);
      Full_Adder FA8(A[7], B[7], c7, S[7], c8);
      Full_Adder FA9(A[8], B[8], c8, S[8], c9);
      Full_Adder FA10(A[9], B[9], c9, S[9], c10);
      Full_Adder FA11(A[10], B[10], c10, S[10], c11);
      Full_Adder FA12(A[11], B[11], c11, S[11], c12);
      Full_Adder FA13(A[12], B[12], c12, S[12], c13);
      Full_Adder FA14(A[13], B[13], c13, S[13], c14);
      Full_Adder FA15(A[14], B[14], c14, S[14], c15);
      Full_Adder FA16(A[15], B[15], c15, S[15], c16);
      Full_Adder FA17(A[16], B[16], c16, S[16], c17);
      Full_Adder FA18(A[17], B[17], c17, S[17], c18);
      Full_Adder FA19(A[18], B[18], c18, S[18], c19);
      Full_Adder FA20(A[19], B[19], c19, S[19], c20);
      Full_Adder FA21(A[20], B[20], c20, S[20], c21);
      Full_Adder FA22(A[21], B[21], c21, S[21], S[22]);


endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
```

```
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    21:06:48 11/28/2008
// Design Name:
// Module Name:    splitter
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: Splits a single bit signal into a 20-bit signal.
//
// Dependencies: none
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module splitter(
    input A,
    output [11:0] S
    );

      assign S = {A,A,A,A,A,A,A,A,A,A,A,A};

endmodule

`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    22:52:14 12/01/2008
// Design Name:
// Module Name:    Adder_13b
// Project Name:   IIR_Filter
// Target Devices:
// Tool versions:
// Description: Adds two 13-bit numbers, resulting in a 14-bit value.
//
// Dependencies: Full_Adder.v
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module Adder_13b(
    input [12:0] A,
    input [12:0] B,
    output [13:0] S
    );

      wire c1, c2, c3, c4, c5, c6, c7, c8, c9, c10, c11, c12;

      Full_Adder FA1(A[0], B[0], 1'b0, S[0], c1);
      Full_Adder FA2(A[1], B[1], c1, S[1], c2);
      Full_Adder FA3(A[2], B[2], c2, S[2], c3);
      Full_Adder FA4(A[3], B[3], c3, S[3], c4);
      Full_Adder FA5(A[4], B[4], c4, S[4], c5);
      Full_Adder FA6(A[5], B[5], c5, S[5], c6);
      Full_Adder FA7(A[6], B[6], c6, S[6], c7);
      Full_Adder FA8(A[7], B[7], c7, S[7], c8);
      Full_Adder FA9(A[8], B[8], c8, S[8], c9);
      Full_Adder FA10(A[9], B[9], c9, S[9], c10);
      Full_Adder FA11(A[10], B[10], c10, S[10], c11);
      Full_Adder FA12(A[11], B[11], c11, S[11], c12);
      Full_Adder FA13(A[12], B[12], c12, S[12], S[13]);

endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:      mvanantwerp@hmc.edu
//
// Create Date:    20:48:16 11/28/2008
// Design Name:
// Module Name:    Full_Adder
// Project Name:
// Target Devices:
// Tool versions:
// Description:  Implements a Full Adder
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
module Full_Adder(
    input A,
    input B,
    input C_in,
    output S,
    output C_out
    );

      assign S = (A^B)^C_in;
      assign C_out = (A&B)||(B&C_in)||(A&C_in);

endmodule
```